MediBook

Doctor Appointment Booking System

PHASE 1: COMPLETE LEARNING GUIDE

Spring Boot REST API · MySQL · JWT Authentication
Production-Grade Architecture

Theory + LLD + Code + Common Mistakes + Real Errors

What Makes This Guide Different

[OK] Why-before-how: theory and design before code

[OK] Low-Level Design diagrams for each module

[OK] SOLID principles and design patterns applied

[OK] All real errors faced during development (and fixes)

[OK] Common mistakes with prevention checklists

[OK] Comparison tables for design decisions

[OK] Complete source code with explanations

[OK] Verification checklist after every step

[OK] Interview talking points

[OK] AWS free-tier strategy

Tech Stack: Java 21 · Spring Boot 3.5 · MySQL 8 · JWT · BCrypt

Build: Maven · IntelliJ IDEA · Postman · Git

Table of Contents

PART I — Project Foundation

1. Project Overview & Business Requirements

MediBook is a doctor appointment booking system. We will build it as a production-grade application following industry standards used by Netflix, Spotify, Uber, and major fintech companies.

1.1 Three User Roles

Role	Capabilities
PATIENT	Register, login, browse doctors, view slots, book appointments, cancel bookings, view history
DOCTOR	Login, view today's bookings, view patient details, mark appointments complete, reschedule
ADMIN	Login, add/edit/delete doctors, configure slot rules (work hours, slot duration, max bookings per slot), view

1.2 Default Business Rules (Admin-Configurable)

Rule	Default Value	Description
Work hours	10:00 - 20:00 IST	Doctor's daily working window
Slot duration	30 minutes	Length of one appointment slot
Max bookings per slot	2	Concurrent patients per slot
Time zone	Asia/Kolkata	All time stored in this TZ
Total slots/day	20	Calculated: (20-10) * 60 / 30
Total bookings/day	40	20 slots * 2 patients per doctor

1.3 Phase 1 Scope

In this Phase 1 guide we cover only the foundation:

- User registration with input validation
- Login with JWT token generation
- Stateless authentication
- Role-based access control
- Global exception handling
- Production-grade project structure

2. Why Monolith-First, Not Microservices-First?

A common beginner mistake is jumping straight to microservices. Industry experts (Martin Fowler, Sam Newman) strongly recommend the opposite approach.

2.1 The Monolith-First Pattern

Build a clean modular monolith first. Once the domain is well understood, split into microservices. The internal package boundaries of a well-structured monolith become natural microservice boundaries later.

2.2 Comparison

Aspect	Microservices-First (Bad)	Monolith-First (Our Way)
Initial complexity	Very high	Low
Network errors	Many (between services)	None initially
Domain understanding	Forced upfront	Emerges naturally
Learning curve	Steep, overwhelming	Gradual, manageable
Debugging	Multi-service tracing	Single process
AWS cost	Many instances	One instance
Refactoring later	Hard (network calls)	Easy (just split packages)

NOTE. Our roadmap: Phase 1 (this guide) = monolith foundation. Later phases will split into auth-service, doctor-service, slot-service, booking-service, notification-service. The package structure we use now is already microservices-ready.

3. Complete Tool Chain & Methodology

3.1 Languages & Core

Tool	Purpose
Java 21 (LTS)	Backend programming language. LTS until 2031.
TypeScript	Frontend language (Phase 7)
SQL	Database queries
YAML	Configuration files (Spring Boot standard)

3.2 Backend Frameworks

Tool	Purpose
Spring Boot 3.5.14	Main framework
Spring Web (MVC)	REST API
Spring Data JPA	Database ORM
Spring Security 6	Authentication & authorization
Hibernate 6	JPA implementation
Bean Validation 3.0	Input validation
HikariCP	Connection pooling (bundled)

3.3 Databases

Tool	Phase	Purpose
MySQL 8	Phase 1+	Primary relational DB
MongoDB	Phase 4+	Booking documents (later)
Redis	Phase 6+	Caching layer
H2	Tests	In-memory for unit tests

3.4 Security & Auth

Tool	Purpose
JJWT 0.12.6	JWT token library
BCrypt	Password hashing (one-way)
Spring Security	Filter chain, role-based access
@PreAuthorize	Method-level security

Tool	Purpose
JWT (RFC 7519)	Stateless tokens

3.5 Testing

Tool	Purpose
JUnit 5	Unit test framework
Mockito	Mocking dependencies
AssertJ	Fluent assertions
REST Assured	API testing
Testcontainers	Integration tests with real DBs
Postman	Manual API testing

3.6 DevOps Tools (Used in Later Phases)

Tool	Phase	Purpose
Maven	All	Build tool
Git + GitHub	All	Version control
Docker	Phase 8	Containerization
Docker Compose	Phase 8	Multi-container dev
GitHub Actions	Phase 9	CI/CD pipeline
Jenkins	Phase 9 bonus	Alternative CI/CD
Kafka (local)	Phase 5	Message broker
AWS SQS	Phase 10	Cloud message queue

3.7 Methodologies & Principles

Principle	Meaning
SOLID	5 OO design principles (we apply all)
DRY	Don't Repeat Yourself
KISS	Keep It Simple, Stupid
YAGNI	You Aren't Gonna Need It
12-Factor App	Cloud-native app methodology
Agile / Scrum	Iterative development

Principle	Meaning
TDD	Test-Driven Development
CI/CD	Continuous Integration & Delivery
Conventional Commits	feat:, fix:, chore: prefixes

4. AWS Free-Tier Strategy

If you have an AWS account, you get the free tier. Understanding what's free vs paid helps you stay safe from surprise bills.

4.1 12-Month Free (from account creation)

Service	Free Limit	What We Use It For
EC2 t2.micro	750 hrs/month	Hosting backend
RDS db.t3.micro	750 hrs/month	MySQL database
ALB	750 hrs + 15 LCUs	Load balancer
S3	5 GB	Frontend build hosting
EBS	30 GB	EC2 storage
Data transfer out	100 GB/month	Outbound traffic

4.2 Always Free (no time limit)

Service	Free Limit
Lambda	1M requests/month + 400,000 GB-sec
API Gateway HTTP API	1M calls/month
DynamoDB	25 GB + 25 RCU/WCU
SNS	1M publishes/month
SQS	1M requests/month
CloudFront	1 TB out/month
CloudWatch	10 metrics + 5 GB logs

4.3 Avoid These (Expensive)

Service	Cost	What to Use Instead
AWS MSK (Kafka)	$100+/month	SQS + SNS (free tier)
EKS (Kubernetes)	$73/month for control plane	ECS Fargate
NAT Gateway	$32/month	Public subnets for learning
Aurora	High	RDS MySQL db.t3.micro
ElastiCache (Redis)	Not free	Run Redis in Docker locally

4.4 Safety Rules

- Set a $5 billing alert in AWS Billing Console BEFORE launching anything
- Stop EC2 / RDS instances when not testing

- Use only t2.micro / db.t3.micro instances
- Max instances = 2 in Auto Scaling Groups
- Delete unused EBS volumes (they keep billing)
- Check Free Tier Dashboard weekly

WARNING. Auto Scaling Groups can launch unlimited EC2s — always cap max=2 for learning. A runaway ASG can burn the 750 free hours in a day.

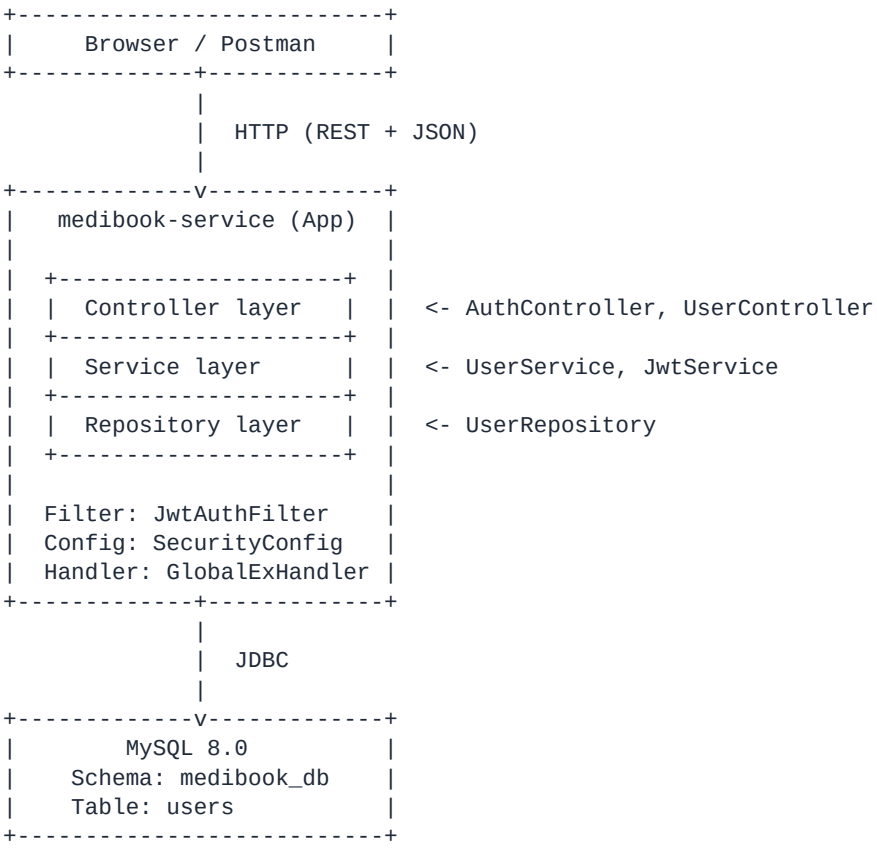
PART II — Architecture & Design

5. High-Level Architecture (HLA)

This is the system's bird's-eye view. Phase 1 builds the monolith on the left side; later phases evolve it into microservices on the right.

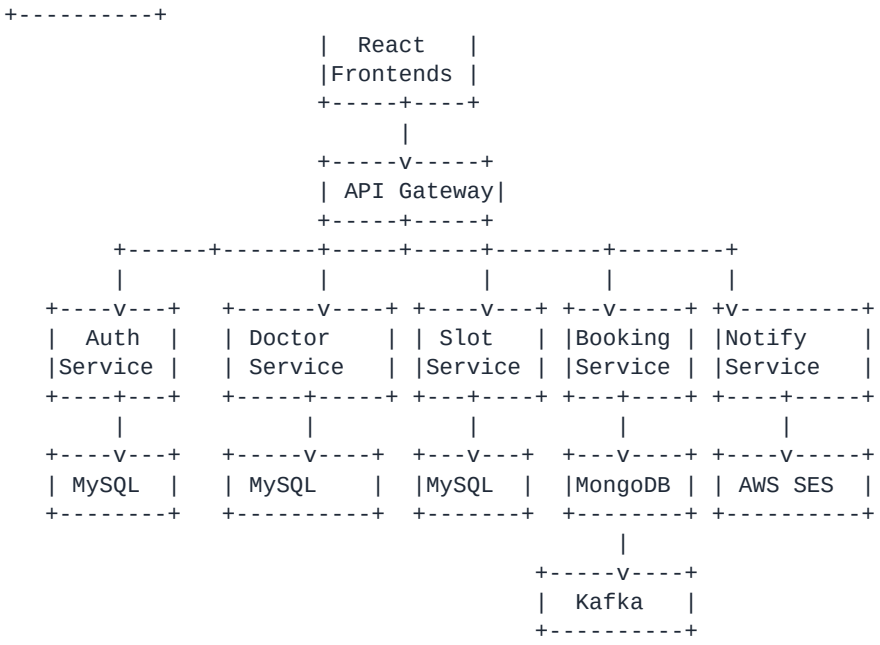
5.1 Phase 1 — Monolithic Architecture

Phase 1 architecture



5.2 Future — Target Microservices Architecture

Phase 6+ target



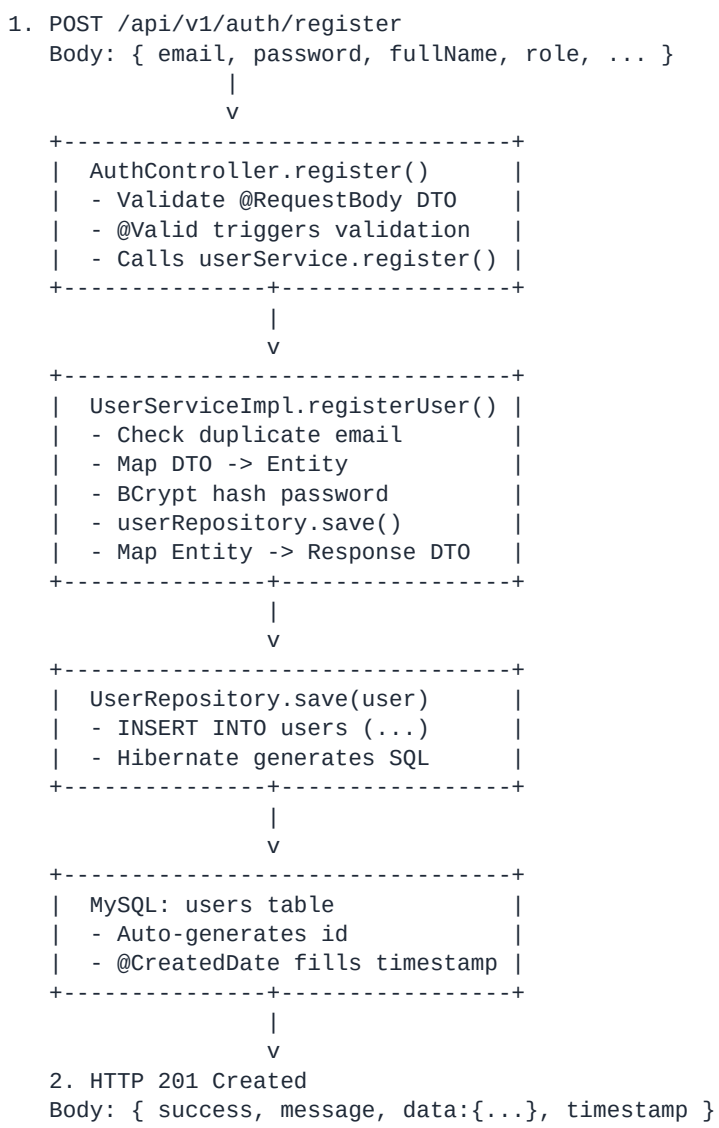
NOTE. Notice: each layer in our Phase 1 monolith maps directly to a service in the target. That is why folder/package structure matters from day one.

6. Low-Level Design (LLD) Overview

LLD shows how a request flows through internal layers, and what each class is responsible for. Below is the LLD for a registration request.

6.1 Layered Request Flow

Registration request LLD



6.2 Class Responsibilities

Class	Responsibility	Annotation
AuthController	HTTP layer; validate input; delegate to service	@RestController
UserService	Business logic contract (interface)	(interface)
UserServiceImpl	Business logic implementation	@Service
UserMapper	Convert between Entity and DTO	@Component
UserRepository	Database access (CRUD); auto-generated by Spring	@Repository
User (Entity)	Maps to users table; defines schema	@Entity
RegisterRequest	Input DTO with validation rules	(POJO)
UserResponse	Output DTO; hides password	(POJO)
GlobalExceptionHandler	Central error handler; returns JSON errors	@RestControllerAdvice

Class	Responsibility	Annotation
JwtService	Generate / validate JWT tokens	@Service
JwtAuthenticationFilter	Intercept requests; validate token; set SecurityContext	@Component
SecurityConfig	Configure Spring Security; define beans	@Configuration

7. SOLID Principles Applied

SOLID is 5 principles for object-oriented design. Every class in MediBook follows them.

S — Single Responsibility

Each class has one reason to change. Examples:

- AuthController only handles HTTP (not business logic)
- UserService only handles business logic (not HTTP or DB)
- UserRepository only handles DB access (no business logic)
- UserMapper only handles Entity-DTO conversion
- GlobalExceptionHandler only handles error responses

O — Open/Closed

Open for extension, closed for modification. Adding new exception handlers does not require modifying existing ones - just add a new @ExceptionHandler method in GlobalExceptionHandler.

L — Liskov Substitution

Subtypes must be substitutable for their parents. Our custom exceptions extend RuntimeException and can be thrown anywhere RuntimeException is expected, without breaking the system.

I — Interface Segregation

Clients should not depend on methods they don't use. UserService interface only has user-related methods. We don't put doctor or booking operations into the same interface.

D — Dependency Inversion

Depend on abstractions, not concretions. AuthController depends on the UserService *interface*, not on UserServiceImpl. This enables easy mocking in tests and swapping implementations later.

Dependency Inversion example

```
// GOOD - depends on interface
private final UserService userService;
public AuthController(UserService userService) {
    this.userService = userService;
}

// BAD - depends on concrete class
private final UserServiceImpl userService;
```

8. Design Patterns Used in Phase 1

Pattern	Category	Where Used	Why
DTO	Architectural	Request/Response classes	Decouple API from DB schema
Mapper	Structural	UserMapper	Centralize Entity-DTO conversion
Repository	Architectural	UserRepository	Abstract database access
Builder	Creational	ErrorResponse.builder()	Construct complex objects step-by-step
Factory Method	Creational	ApiResponse.success() / .error()	Centralized object creation

Pattern	Category	Where Used	Why
Dependency Injection	Creational	All Spring components	Loose coupling, testability
Filter Chain	Behavioral	JwtAuthenticationFilter	Cross-cutting concerns (auth)
Singleton	Creational	All @Service / @Component beans	One instance per app context
Strategy	Behavioral	PasswordEncoder (interface)	Swappable hashing algorithms

NOTE. We will add more patterns in later phases: CQRS (booking command/query), Circuit Breaker (Resilience4j), API Gateway pattern, Saga (distributed transactions).

PART III — Environment Setup

9. Install Java 21, Maven, MySQL, IntelliJ, Git

9.1 Java 21 (LTS)

Download Eclipse Temurin JDK 21 from *https://adoptium.net/*

During Windows install, tick: Set JAVA_HOME · Add to PATH · JavaSoft (Oracle) registry keys

Verify

```
java -version
# Expected: openjdk version "21.0.x"
```

WARNING. Java 21 is LTS until 2031. Avoid Java 24 (too new, library issues) and Java 8 (too old, unsupported features). Spring Boot 3.5 needs Java 17+.

9.2 Maven

Download binary zip from *https://maven.apache.org/download.cgi*

Extract to **C:\Program Files\Maven**, add the **bin** folder to PATH

Verify

```
mvn -version
# Expected: Apache Maven 3.9.x
```

9.3 MySQL 8.0

Install MySQL Installer (Developer Default). Set a strong root password.

Run in MySQL Workbench

```
CREATE DATABASE medibook_db;
```

9.4 IntelliJ IDEA Community Edition

Free from JetBrains. Configure JDK: File » Project Structure » SDKs

9.5 Git + GitHub account

Install from *https://git-scm.com/download/win*. Create a free GitHub account.

TIP. We use Git/GitHub instead of AWS CodeCommit because Git/GitHub is the industry standard, has the largest community, and integrates with all CI/CD tools.

10. Common Setup Mistakes & Fixes

Mistake / Symptom	Cause	Fix
'java' is not recognized	JAVA_HOME not in PATH	Add JDK bin folder to PATH; reopen terminal
'mvn' is not recognized	Maven not in PATH	Add Maven bin folder to PATH; reopen terminal
Multiple Java versions found	System has Java 8 + 21	Remove old Java from PATH; set JAVA_HOME to 21
IntelliJ shows wrong Java	Project SDK misconfigured	File > Project Structure > set SDK to 21
MySQL connection refused	MySQL service not running	Start MySQL service in Windows Services
MySQL access denied	Wrong password	Reconfigure via MySQL Installer or reset root password

Mistake / Symptom	Cause	Fix
Public Key Retrieval not allowed	MySQL 8 auth issue	Add allowPublicKeyRetrieval=true to JDBC URL
Port 8080 already in use	Previous Java process running	Kill process: netstat -ano | findstr :8080 then taskkill /PID xx
IntelliJ Lombok plugin install fails	Corporate firewall blocks marketplace	Use plain Java getters/setters instead
YAML parser error	Tabs instead of spaces, or wrong indent	Use 2 spaces only; verify indentation

CRITICAL. Real error we hit: 'Public Key Retrieval is not allowed'. Fix: Add **allowPublicKeyRetrieval=true** to the JDBC URL in application-dev.yml. This is required for MySQL 8 caching_sha2_password authentication.

PART IV — Project Initialization

11. Spring Initializr Configuration

Open *https://start.spring.io* and fill exactly:

Field	Value
Project	Maven
Language	Java
Spring Boot	3.5.14 (highest stable, no SNAPSHOT/RC)
Group	com.medibook
Artifact	medibook-service
Name	medibook-service
Description	Doctor Appointment Booking System
Package name	com.medibook.medibookservice
Packaging	Jar
Java	21

11.1 Dependencies to Add

Dependency	Why we need it
Spring Web	Build REST APIs
Spring Data JPA	Database ORM
MySQL Driver	MySQL JDBC connector
Spring Security	Auth & authorization
Validation	Bean Validation annotations
Spring Boot DevTools	Auto-restart on code changes
Spring Boot Actuator	Production monitoring (health checks)

WARNING. Avoid Spring Boot 4.x.x (too new). Avoid anything labeled SNAPSHOT or RC. Use the highest stable 3.5.x available.

11.2 Open in IntelliJ

- Click Generate » download zip
- Extract to **C:\Projects\medibook-service**
- File » Open » select that folder » Trust Project
- Wait 2-5 minutes for Maven to download dependencies

12. Production-Grade Folder Structure (LLD)

Folder structure is part of LLD - it dictates how modules will split later. We create these packages BEFORE writing any code, so each new class has a clear home.

Project structure

```
medibook-service/
|
+-- src/
|   +-- main/
|   |   +-- java/com/medibook/medibookservice/
|   |   |   |
|   |   |   +-- MedibookServiceApplication.java   <- Main class
|   |   |   |
|   |   |   +-- config/         <- Spring config (SecurityConfig)
|   |   |   +-- controller/     <- REST endpoints
|   |   |   +-- service/        <- Business logic interfaces
|   |   |   |   +-- impl/       <- Service implementations
|   |   |   +-- repository/     <- JPA repositories
|   |   |   +-- entity/         <- JPA entities (DB tables)
|   |   |   +-- dto/
|   |   |   |   +-- request/    <- Incoming API DTOs
|   |   |   |   +-- response/   <- Outgoing API DTOs
|   |   |   +-- mapper/         <- Entity <-> DTO converters
|   |   |   +-- exception/      <- Custom exceptions + handler
|   |   |   +-- security/       <- JWT, filters, user details
|   |   |   +-- enums/          <- Role and other enums
|   |   |   +-- constants/      <- App-wide constants
|   |   |   +-- util/           <- Helper classes
|   |   |
|   |   +-- resources/
|   |       +-- application.yml         <- Common config
|   |       +-- application-dev.yml     <- Dev environment
|   |       +-- application-prod.yml    <- Production env
|   |
|   +-- test/
|       +-- java/...
|
+-- pom.xml
+-- .gitignore
```

TIP. When we split into microservices in Phase 6, each **top-level package** (controller, service, repository, entity for one domain) becomes a separate service. This structure is already microservices-ready.

13. YAML Configuration with Profiles

13.1 Why YAML over .properties?

application.properties (Old)	application.yml (Modern)
Flat, repetitive	Hierarchical, clean
spring.datasource.url=...	Nested structure
Hard to read with many configs	Industry standard for Spring Boot
Limited support for lists/maps	Native support

13.2 Why Profiles?

Same code, different settings per environment. Examples:

- dev profile: local MySQL, verbose logging, auto-create tables
- staging profile: AWS test DB, moderate logging
- prod profile: AWS production DB, minimal logging, ddl validate only

13.3 application.yml (common config)

src/main/resources/application.yml

```yaml
spring:
  application:
    name: medibook-service
  profiles:
    active: dev                         # Default profile
  jpa:
    open-in-view: false
    properties:
      hibernate:
        format_sql: true
        use_sql_comments: true

server:
  port: 8080
  error:
    include-message: always
    include-binding-errors: always
  servlet:
    context-path: /api/v1               # All APIs prefixed with /api/v1

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      show-details: always
      show-components: always

info:
  app:
    name: MediBook
    description: Doctor Appointment Booking System
    version: 1.0.0

medibook:
  booking:
    default-work-start: "10:00"
    default-work-end: "20:00"
    default-slot-duration-minutes: 30
    default-max-bookings-per-slot: 2
    timezone: "Asia/Kolkata"
  jwt:
    secret: ${JWT_SECRET:medibook-super-secret-key-for-jwt-signing-2026-must-be-long-enough}
    expiration-ms: 86400000             # 24 hours

logging:
  level:
    root: INFO
    com.medibook: DEBUG
```

13.4 application-dev.yml (local dev)

src/main/resources/application-dev.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/medibook_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyR
    username: ${DB_USERNAME:root}
    password: ${DB_PASSWORD}
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 5
      connection-timeout: 30000

  jpa:
    hibernate:
      ddl-auto: update                # Auto-create/update tables in dev
    show-sql: true
    database-platform: org.hibernate.dialect.MySQLDialect

logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE
```

13.5 application-prod.yml (AWS production)

src/main/resources/application-prod.yml

```
spring:
  datasource:
    url: ${DB_URL}                    # From AWS environment
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      maximum-pool-size: 20

  jpa:
    hibernate:
      ddl-auto: validate              # NEVER auto-modify in production!
    show-sql: false

logging:
  level:
    root: WARN
    com.medibook: INFO
```

13.6 YAML Golden Rules

- Use **2 spaces** for indentation. Never tabs.
- Same indent level = same hierarchy level
- Space after colon: **key: value**, not **key:value**
- Root keys (spring:, server:, logging:) at column 1
- Strings with special chars need quotes: "10:00"

CRITICAL. Real error we hit: YAML parser error. The cause was that **logging:** was indented under **spring:** by accident, making YAML think it was a nested setting. Lesson learned: indentation in YAML is meaningful - it defines hierarchy.

13.7 Secure the DB Password

Never hard-code passwords. Notice **${DB_PASSWORD}** in YAML - this reads from an environment variable. Set it in IntelliJ:

- Run Configurations dropdown » Edit Configurations
- Modify options » Environment variables (enable)
- Add: **DB_PASSWORD=your_mysql_password** (no quotes, no spaces)

WARNING. Common mistake: writing **DB_PASSWORD="root1234"** with quotes. Don't use quotes — just **DB_PASSWORD=root1234**.

PART V — User Module (Deep Dive)

14. Module LLD & Layered Flow

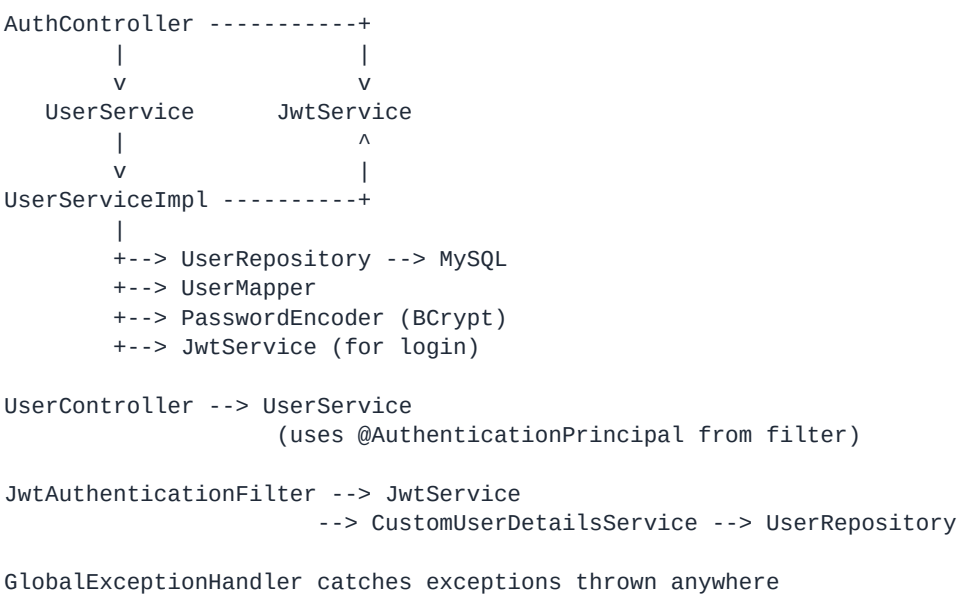
Before writing code, let's understand the complete layout of the User module. This is the foundation — every other module follows this same pattern.

14.1 What We'll Build (in order)

#	Component	Type
1	Role enum	Enum
2	User entity	JPA Entity
3	UserRepository	Spring Data interface
4	DTOs (Register/Login Request, User/Auth/Api/Error Response)	POJOs
5	Custom Exceptions (3)	RuntimeException subclasses
6	GlobalExceptionHandler	@RestControllerAdvice
7	UserMapper	@Component
8	UserService (interface) + UserServiceImpl	Service layer
9	SecurityConfig (initial - permitAll)	@Configuration
10	AuthController (register only)	@RestController
11	JwtService (after login basics work)	@Service
12	AuthController (add login)	REST endpoint
13	JwtAuthenticationFilter	OncePerRequestFilter
14	CustomUserDetailsService	Spring Security
15	SecurityConfig (full — JWT-aware)	@Configuration
16	UserController (protected /me, /admin-only)	REST

14.2 Dependency Graph

Dependency graph



15. Role Enum

15.1 Why an Enum, Not a String?

String (bad)	Enum (good)
String role = "Patient";	Role role = Role.PATIENT;
Typos: "patient" vs "Patient"	Compiler catches typos
Need manual validation	Type-safe automatically
Hard to refactor	IDE renames everywhere
No autocomplete	IDE autocomplete works

15.2 Code

src/main/java/com/medibook/medibookservice/enums/Role.java

```
package com.medibook.medibookservice.enums;

/**
 * Defines user roles in the system.
 * Used for role-based access control (RBAC).
 */
public enum Role {
    PATIENT,
    DOCTOR,
    ADMIN
}
```

NOTE. Storing as STRING in DB: We use @Enumerated(EnumType.STRING) on the User entity. This saves "PATIENT" in the DB instead of 0. Far easier to debug; resilient to enum reordering.

16. User Entity (JPA + Auditing + Indexing)

16.1 Concept: What is a JPA Entity?

A JPA Entity is a Java class that maps to a database table. Hibernate (the JPA implementation) auto-creates the table from the class when **ddl-auto: update** is set in dev profile.

Mapping at a glance:

Java field	DB column (auto-derived)	Type mapping
id	id	BIGINT primary key auto-increment
email	email	VARCHAR(100) unique not null
password	password	VARCHAR(255) not null
fullName	full_name	VARCHAR(100) not null
phoneNumber	phone_number	VARCHAR(15)
role	role	VARCHAR(20) stored as enum string
enabled	enabled	BOOLEAN

Java field	DB column (auto-derived)	Type mapping
createdAt	created_at	DATETIME auto-filled
updatedAt	updated_at	DATETIME auto-updated

16.2 Why we Index the email Column

Every login executes **SELECT * FROM users WHERE email = ?**. Without an index, MySQL scans every row. With an index, MySQL jumps directly to the matching row.

Number of Users	Without Index	With Index
1,000	~10 ms	~1 ms
100,000	~500 ms	~1 ms
1,000,000	~5,000 ms (5 sec!)	~2 ms

TIP. Indexes do NOT create a separate table. They are a sorted lookup structure inside the same table. Think of them like the index at the back of a book - still part of the book, just sorted for quick lookup.

16.3 Full Code (Plain Java, no Lombok)

src/main/java/com/medibook/medibookservice/entity/User.java

```
package com.medibook.medibookservice.entity;

import com.medibook.medibookservice.enums.Role;
import jakarta.persistence.*;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.LocalDateTime;

@Entity
@Table(name = "users", indexes = {
        @Index(name = "idx_user_email", columnList = "email", unique = true)
})
@EntityListeners(AuditingEntityListener.class)
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "email", nullable = false, unique = true, length = 100)
    private String email;

    @Column(name = "password", nullable = false)
    private String password;

    @Column(name = "full_name", nullable = false, length = 100)
    private String fullName;

    @Column(name = "phone_number", length = 15)
    private String phoneNumber;

    @Enumerated(EnumType.STRING)
    @Column(name = "role", nullable = false, length = 20)
    private Role role;

    @Column(name = "enabled", nullable = false)
    private boolean enabled = true;

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    public User() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public String getPhoneNumber() { return phoneNumber; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }
```

```
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }

    public boolean isEnabled() { return enabled; }
    public void setEnabled(boolean enabled) { this.enabled = enabled; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
}
```

16.4 Annotations Reference

Annotation	Purpose
@Entity	Marks the class as a JPA entity (DB table)
@Table(name=...)	Specifies the actual table name
@Id	Marks the primary key field
@GeneratedValue(IDENTITY)	Auto-increment ID (DB generates it)
@Column	Customize column (length, nullable, unique)
@Enumerated(EnumType.STRING)	Save enum as text "PATIENT" not number 0
@Index(unique=true)	Named DB index for fast lookups
@CreatedDate	Auto-fill creation timestamp
@LastModifiedDate	Auto-update on every save
@EntityListeners(AuditingEntityListener.class)	Enables JPA auditing for this entity

16.5 Enable JPA Auditing

For @CreatedDate and @LastModifiedDate to work, add **@EnableJpaAuditing** to the main application class:

MedibookServiceApplication.java

```
package com.medibook.medibookservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@SpringBootApplication
@EnableJpaAuditing       // <-- ADD THIS
public class MedibookServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(MedibookServiceApplication.class, args);
    }
}
```

16.6 Why We Dropped Lombok (Real Story)

We initially tried **Lombok** annotations (@Getter, @Setter, @Builder) to reduce boilerplate. On the corporate laptop, the IntelliJ Lombok plugin failed to install with **'Request failed with status code 40x'** -

the marketplace was blocked by the corporate firewall.

Lombok (~5 annotations)	Plain Java (manual)
@Getter @Setter @NoArgsConstructor @Builder	Write getters/setters/constructors manually
~15 lines per class	~80 lines per class
Needs plugin installed	Works with any IDE
Needs annotation processing enabled	No special setup needed
Less Java code visible	More transparent for learning

TIP. Lesson learned: in restricted environments, Lombok is risky. Plain Java is verbose but always works. IntelliJ can generate getters/setters: **Alt+Insert » Getters and Setters**.

17. UserRepository (Spring Data Magic)

17.1 What Spring Data JPA Does for You

By extending **JpaRepository<Entity, IdType>**, you get these methods for free with auto-generated SQL:

Method	Auto-generated SQL
save(user)	INSERT or UPDATE based on id
findById(id)	SELECT * FROM users WHERE id = ?
findAll()	SELECT * FROM users
findAll(Pageable)	SELECT ... LIMIT ? OFFSET ?
deleteById(id)	DELETE FROM users WHERE id = ?
count()	SELECT COUNT(*) FROM users
existsById(id)	SELECT 1 FROM users WHERE id = ? LIMIT 1

17.2 Derived Query Methods

Spring reads method names and generates SQL automatically. Examples:

Method Name	Generated SQL
findByEmail(String email)	SELECT * FROM users WHERE email = ?
existsByEmail(String email)	SELECT 1 FROM users WHERE email = ? LIMIT 1
findByRole(Role role)	SELECT * FROM users WHERE role = ?
findByEmailAndEnabled(...)	SELECT * FROM users WHERE email = ? AND enabled = ?
countByRole(Role role)	SELECT COUNT(*) FROM users WHERE role = ?
findByEmailContainingIgnoreCase(...)	SELECT * FROM users WHERE LOWER(email) LIKE LOWER(?)

17.3 Code

src/main/java/com/medibook/medibookservice/repository/UserRepository.java

```
package com.medibook.medibookservice.repository;

import com.medibook.medibookservice.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

/**
 * Repository for User entity.
 * Spring Data JPA auto-generates implementations at runtime.
 */
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);
}
```

17.4 Why Optional Instead of null?

Optional<User> forces the caller to handle the 'not found' case explicitly. This is a modern Java best practice that prevents NullPointerException.

Working with Optional

```
// Caller code
Optional<User> userOpt = userRepository.findByEmail("john@example.com");

// Option 1: use orElseThrow
User user = userRepository.findByEmail(email)
    .orElseThrow(() -> new ResourceNotFoundException("User", "email", email));

// Option 2: use ifPresent
userOpt.ifPresent(user -> System.out.println(user.getName()));

// Option 3: get with default
User user = userOpt.orElse(new User());
```

18. DTOs — Why and How

18.1 The Problem with Using Entities Directly

Consider this bad code: *return userRepository.save(user);* as REST response. Problems:

- **Password leaked:** API response includes hashed password
- **Internal fields exposed:** createdAt, updatedAt, enabled
- **Mass assignment attack:** Hacker sends role=ADMIN in body and becomes admin
- **Tight coupling:** DB schema changes break API clients
- **No API-specific validation:** Email must be strong for API; not always for DB

18.2 The DTO Solution

DTO flow

```
Client sends JSON
          |
          v
   RegisterRequest DTO         <-- @Valid annotations check input
   { email, password,              { @NotBlank, @Email, @Pattern }
     fullName, role, ... }
          |
          | (UserMapper.toEntity)
          v
   User Entity                 <-- For DB only
   { id, email, password,          (password hashed here)
     role, enabled, dates }
          |
          | (UserMapper.toResponse)
          v
   UserResponse DTO            <-- NO password field!
   { id, email, fullName,          (safe to send to client)
     phoneNumber, role,
     enabled, createdAt }
```

18.3 RegisterRequest with Validation

dto/request/RegisterRequest.java

```
package com.medibook.medibookservice.dto.request;

import com.medibook.medibookservice.enums.Role;
import jakarta.validation.constraints.*;

public class RegisterRequest {

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    @Size(max = 100, message = "Email cannot exceed 100 characters")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, max = 50, message = "Password must be 8-50 chars")
    @Pattern(
        regexp = "^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*#?&])[A-Za-z\d@$!%*#?&]{8,}$",
        message = "Password must contain at least one letter, one number, and one special character"
    )
    private String password;

    @NotBlank(message = "Full name is required")
    @Size(min = 2, max = 100)
    private String fullName;

    @Pattern(regexp = "^[+]?[0-9]{10,15}$", message = "Invalid phone number format")
    private String phoneNumber;

    @NotNull(message = "Role is required")
    private Role role;

    public RegisterRequest() {}

    // getters and setters
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public String getPhoneNumber() { return phoneNumber; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }
}
```

18.4 Validation Annotations Reference

Annotation	What it Checks
@NotNull	Not null (empty string OK)
@NotBlank	Not null AND not empty ("" or " " rejected)
@NotEmpty	Not null AND has content (lists/maps/strings)
@Email	Valid email format
@Size(min, max)	String length, collection size
@Min / @Max	Numeric range
@Pattern(regexp)	Custom regex

Annotation	What it Checks
@Past / @Future	Date validation
@Positive / @Negative	Numeric sign

18.5 LoginRequest

dto/request/LoginRequest.java

```
package com.medibook.medibookservice.dto.request;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public class LoginRequest {

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Password is required")
    private String password;

    public LoginRequest() {}

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

18.6 UserResponse (no password!)

dto/response/UserResponse.java

```
package com.medibook.medibookservice.dto.response;

import com.medibook.medibookservice.enums.Role;
import java.time.LocalDateTime;

public class UserResponse {

    private Long id;
    private String email;
    private String fullName;
    private String phoneNumber;
    private Role role;
    private boolean enabled;
    private LocalDateTime createdAt;

    public UserResponse() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public String getPhoneNumber() { return phoneNumber; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }
    public boolean isEnabled() { return enabled; }
    public void setEnabled(boolean enabled) { this.enabled = enabled; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
}
```

18.7 AuthResponse (login response with JWT)

dto/response/AuthResponse.java

```
package com.medibook.medibookservice.dto.response;

import com.medibook.medibookservice.enums.Role;

public class AuthResponse {

    private String token;
    private String tokenType;     // "Bearer"
    private Long userId;
    private String email;
    private String fullName;
    private Role role;

    public AuthResponse() {}

    public AuthResponse(String token, String tokenType, Long userId,
                        String email, String fullName, Role role) {
        this.token = token;
        this.tokenType = tokenType;
        this.userId = userId;
        this.email = email;
        this.fullName = fullName;
        this.role = role;
    }

    // getters and setters
    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }
    public String getTokenType() { return tokenType; }
    public void setTokenType(String tokenType) { this.tokenType = tokenType; }
    public Long getUserId() { return userId; }
    public void setUserId(Long userId) { this.userId = userId; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }
}
```

18.8 ApiResponse (generic envelope with Factory pattern)

We wrap every response in a consistent envelope. Notice the static factory methods **success()** and **error()** - this is the Factory Method pattern.

dto/response/ApiResponse.java

```
package com.medibook.medibookservice.dto.response;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.time.LocalDateTime;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ApiResponse<T> {

    private boolean success;
    private String message;
    private T data;
    private LocalDateTime timestamp;

    public ApiResponse() {}

    public ApiResponse(boolean success, String message, T data, LocalDateTime timestamp) {
        this.success = success;
        this.message = message;
        this.data = data;
        this.timestamp = timestamp;
    }

    // Factory methods
    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(true, message, data, LocalDateTime.now());
    }

    public static <T> ApiResponse<T> error(String message) {
        return new ApiResponse<>(false, message, null, LocalDateTime.now());
    }

    // getters and setters
    public boolean isSuccess() { return success; }
    public void setSuccess(boolean success) { this.success = success; }
    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }
    public T getData() { return data; }
    public void setData(T data) { this.data = data; }
    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp; }
}
```

18.9 ErrorResponse (with Builder pattern)

ErrorResponse has many optional fields (path, validationErrors). The Builder pattern makes construction clean and readable.

dto/response/ErrorResponse.java

```
package com.medibook.medibookservice.dto.response;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.time.LocalDateTime;
import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private boolean success;
    private int status;
    private String error;
    private String message;
    private String path;
    private LocalDateTime timestamp;
    private Map<String, String> validationErrors;

    public ErrorResponse() {}

    private ErrorResponse(Builder b) {
        this.success = b.success;
        this.status = b.status;
        this.error = b.error;
        this.message = b.message;
        this.path = b.path;
        this.timestamp = b.timestamp;
        this.validationErrors = b.validationErrors;
    }

    public static Builder builder() { return new Builder(); }

    public static class Builder {
        private boolean success;
        private int status;
        private String error;
        private String message;
        private String path;
        private LocalDateTime timestamp;
        private Map<String, String> validationErrors;

        public Builder success(boolean s)       { this.success = s; return this; }
        public Builder status(int s)            { this.status = s; return this; }
        public Builder error(String e)          { this.error = e; return this; }
        public Builder message(String m)        { this.message = m; return this; }
        public Builder path(String p)           { this.path = p; return this; }
        public Builder timestamp(LocalDateTime t) { this.timestamp = t; return this; }
        public Builder validationErrors(Map<String, String> v) {
            this.validationErrors = v; return this;
        }
        public ErrorResponse build() { return new ErrorResponse(this); }
    }

    // getters and setters omitted for brevity (generate with IntelliJ)
}
```

19. Custom Exceptions + Global Handler

19.1 Why Not Just RuntimeException?

Generic exceptions all map to HTTP 500. We need different status codes for different problems (404 not found, 409 conflict, 400 bad request).

Bad	Good
throw new RuntimeException("Not found")	throw new ResourceNotFoundException("User", "email", email)
Cannot differentiate types	Clear meaning
Always returns 500	Maps to 404/409/400
Generic message	Structured message

19.2 ResourceNotFoundException

exception/ResourceNotFoundException.java

```
package com.medibook.medibookservice.exception;

public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }

    public ResourceNotFoundException(String resource, String field, Object value) {
        super(String.format("%s not found with %s: '%s'", resource, field, value));
    }
}
```

19.3 ResourceAlreadyExistsException

exception/ResourceAlreadyExistsException.java

```
package com.medibook.medibookservice.exception;

public class ResourceAlreadyExistsException extends RuntimeException {

    public ResourceAlreadyExistsException(String message) {
        super(message);
    }

    public ResourceAlreadyExistsException(String resource, String field, Object value) {
        super(String.format("%s already exists with %s: '%s'", resource, field, value));
    }
}
```

19.4 BadRequestException

exception/BadRequestException.java

```
package com.medibook.medibookservice.exception;

public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}
```

19.5 GlobalExceptionHandler

Central place to catch all exceptions thrown in any controller and return standardized JSON. Annotated with @**RestControllerAdvice**.

exception/GlobalExceptionHandler.java

```
package com.medibook.medibookservice.exception;

import com.medibook.medibookservice.dto.response.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFound(
            ResourceNotFoundException ex, HttpServletRequest request) {
        log.warn("Resource not found: {}", ex.getMessage());
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), request, null);
    }

    @ExceptionHandler(ResourceAlreadyExistsException.class)
    public ResponseEntity<ErrorResponse> handleAlreadyExists(
            ResourceAlreadyExistsException ex, HttpServletRequest request) {
        log.warn("Resource exists: {}", ex.getMessage());
        return build(HttpStatus.CONFLICT, ex.getMessage(), request, null);
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ErrorResponse> handleBadRequest(
            BadRequestException ex, HttpServletRequest request) {
        log.warn("Bad request: {}", ex.getMessage());
        return build(HttpStatus.BAD_REQUEST, ex.getMessage(), request, null);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex, HttpServletRequest request) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach(err -> {
            String field = ((FieldError) err).getField();
            errors.put(field, err.getDefaultMessage());
        });
        log.warn("Validation failed: {}", errors);
        return build(HttpStatus.BAD_REQUEST,
            "Input validation failed for one or more fields", request, errors);
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<ErrorResponse> handleBadCredentials(
            BadCredentialsException ex, HttpServletRequest request) {
        log.warn("Auth failed: {}", ex.getMessage());
        return build(HttpStatus.UNAUTHORIZED,
            "Invalid email or password", request, null);
```

```
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> handleAccessDenied(
            AccessDeniedException ex, HttpServletRequest request) {
        log.warn("Access denied: {}", ex.getMessage());
        return build(HttpStatus.FORBIDDEN,
            "You don't have permission to access this resource", request, null);
    }

    // Catch-all (must be last)
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(
            Exception ex, HttpServletRequest request) {
        log.error("Unhandled exception", ex);
        return build(HttpStatus.INTERNAL_SERVER_ERROR,
            "An unexpected error occurred. Please try again later.",
            request, null);
    }

    private ResponseEntity<ErrorResponse> build(HttpStatus status, String message,
            HttpServletRequest request, Map<String, String> validationErrors) {
        ErrorResponse e = ErrorResponse.builder()
                .success(false)
                .status(status.value())
                .error(status.getReasonPhrase())
                .message(message)
                .path(request.getRequestURI())
                .timestamp(LocalDateTime.now())
                .validationErrors(validationErrors)
                .build();
        return new ResponseEntity<>(e, status);
    }
}
```

19.6 HTTP Status Mapping Reference

Exception	HTTP Status	When
ResourceNotFoundException	404 Not Found	Looking up missing entity
ResourceAlreadyExistsException	409 Conflict	Duplicate email on register
BadRequestException	400 Bad Request	Malformed input
MethodArgumentNotValidException	400 Bad Request	@Valid failed
BadCredentialsException	401 Unauthorized	Login failed
AccessDeniedException	403 Forbidden	Wrong role
Exception (catch-all)	500 Server Error	Anything unhandled

TIP. Security best practice: the catch-all handler logs the full stack trace internally but returns a generic message to the client. This prevents leaking internals to attackers.

20. UserMapper (Mapper Pattern)

The Mapper centralizes Entity <-> DTO conversion in one place. Follows DRY principle - no duplicated conversion logic across the codebase.

mapper/UserMapper.java

```
package com.medibook.medibookservice.mapper;

import com.medibook.medibookservice.dto.request.RegisterRequest;
import com.medibook.medibookservice.dto.response.UserResponse;
import com.medibook.medibookservice.entity.User;
import org.springframework.stereotype.Component;

@Component
public class UserMapper {

    public User toEntity(RegisterRequest request) {
        if (request == null) return null;
        User user = new User();
        user.setEmail(request.getEmail());
        user.setPassword(request.getPassword());     // will be hashed in service
        user.setFullName(request.getFullName());
        user.setPhoneNumber(request.getPhoneNumber());
        user.setRole(request.getRole());
        user.setEnabled(true);
        return user;
    }

    public UserResponse toResponse(User user) {
        if (user == null) return null;
        UserResponse response = new UserResponse();
        response.setId(user.getId());
        response.setEmail(user.getEmail());
        response.setFullName(user.getFullName());
        response.setPhoneNumber(user.getPhoneNumber());
        response.setRole(user.getRole());
        response.setEnabled(user.isEnabled());
        response.setCreatedAt(user.getCreatedAt());
        return response;
    }
}
```

NOTE. Alternative: MapStruct library auto-generates mapping code from interfaces. We use manual mappers for transparency - you can see exactly what's converted.

21. UserService (Interface + Impl)

21.1 Why Separate Interface from Implementation?

- **Dependency Inversion:** Controllers depend on the interface, not the impl
- **Testability:** Easy to mock the interface in unit tests
- **Swap implementations:** Could have different impls for different profiles
- **Industry standard:** Every production Spring Boot project does this

21.2 UserService Interface

service/UserService.java

```
package com.medibook.medibookservice.service;

import com.medibook.medibookservice.dto.request.LoginRequest;
import com.medibook.medibookservice.dto.request.RegisterRequest;
import com.medibook.medibookservice.dto.response.AuthResponse;
import com.medibook.medibookservice.dto.response.UserResponse;
import com.medibook.medibookservice.entity.User;

public interface UserService {

    UserResponse registerUser(RegisterRequest request);

    AuthResponse login(LoginRequest request);

    User findByEmail(String email);

    boolean existsByEmail(String email);
}
```

21.3 UserServiceImpl

Notes about the implementation below:

- **Constructor injection** with final fields - modern best practice
- **No @Autowired** needed - Spring detects single constructor
- **@Transactional** on writes, **@Transactional(readOnly=true)** on reads
- **passwordEncoder.encode()** hashes with BCrypt before save
- **passwordEncoder.matches()** compares hash on login
- Same error "Invalid email or password" for wrong email AND wrong password (OWASP)

service/impl/UserServiceImpl.java

```
package com.medibook.medibookservice.service.impl;

import com.medibook.medibookservice.dto.request.LoginRequest;
import com.medibook.medibookservice.dto.request.RegisterRequest;
import com.medibook.medibookservice.dto.response.AuthResponse;
import com.medibook.medibookservice.dto.response.UserResponse;
import com.medibook.medibookservice.entity.User;
import com.medibook.medibookservice.exception.ResourceAlreadyExistsException;
import com.medibook.medibookservice.exception.ResourceNotFoundException;
import com.medibook.medibookservice.mapper.UserMapper;
import com.medibook.medibookservice.repository.UserRepository;
import com.medibook.medibookservice.security.JwtService;
import com.medibook.medibookservice.service.UserService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class UserServiceImpl implements UserService {

    private static final Logger log = LoggerFactory.getLogger(UserServiceImpl.class);

    private final UserRepository userRepository;
    private final UserMapper userMapper;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public UserServiceImpl(UserRepository userRepository, UserMapper userMapper,
                           PasswordEncoder passwordEncoder, JwtService jwtService) {
        this.userRepository = userRepository;
        this.userMapper = userMapper;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    @Override
    @Transactional
    public UserResponse registerUser(RegisterRequest request) {
        log.info("Registering user: {}", request.getEmail());

        if (userRepository.existsByEmail(request.getEmail())) {
            throw new ResourceAlreadyExistsException("User", "email", request.getEmail());
        }

        User user = userMapper.toEntity(request);
        user.setPassword(passwordEncoder.encode(request.getPassword()));   // BCrypt!
        User saved = userRepository.save(user);

        log.info("User registered with id: {}", saved.getId());
        return userMapper.toResponse(saved);
    }

    @Override
    @Transactional(readOnly = true)
    public AuthResponse login(LoginRequest request) {
        log.info("Login attempt: {}", request.getEmail());

        User user = userRepository.findByEmail(request.getEmail())
                .orElseThrow(() -> new BadCredentialsException("Invalid email or password"));

        if (!user.isEnabled()) {
```

```
            throw new BadCredentialsException("Account is disabled");
        }

        if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
            throw new BadCredentialsException("Invalid email or password");
        }

        String token = jwtService.generateToken(user);
        log.info("Login successful: {}", request.getEmail());

        return new AuthResponse(token, "Bearer", user.getId(),
                user.getEmail(), user.getFullName(), user.getRole());
    }

    @Override
    @Transactional(readOnly = true)
    public User findByEmail(String email) {
        return userRepository.findByEmail(email)
                .orElseThrow(() -> new ResourceNotFoundException("User", "email", email));
    }

    @Override
    @Transactional(readOnly = true)
    public boolean existsByEmail(String email) {
        return userRepository.existsByEmail(email);
    }
}
```

WARNING. Security note: We return the SAME error message for 'user not found' and 'wrong password'. Otherwise hackers could test 1000 emails and discover which ones exist (email enumeration attack).

22. AuthController (Registration Endpoint)

The controller is the thinnest layer - it just receives HTTP requests, calls the service, and returns responses. No business logic.

22.1 Annotations Used

Annotation	Purpose
@RestController	REST endpoint controller; auto-converts return to JSON
@RequestMapping("/auth")	Base path for all methods in class
@PostMapping("/register")	Handles POST /auth/register
@RequestBody	Convert incoming JSON to Java object
@Valid	Trigger validation (@NotBlank, @Email, etc.)
ResponseEntity<T>	Lets you control HTTP status, body, and headers

22.2 Initial AuthController (registration only)

controller/AuthController.java

```
package com.medibook.medibookservice.controller;

import com.medibook.medibookservice.dto.request.LoginRequest;
import com.medibook.medibookservice.dto.request.RegisterRequest;
import com.medibook.medibookservice.dto.response.ApiResponse;
import com.medibook.medibookservice.dto.response.AuthResponse;
import com.medibook.medibookservice.dto.response.UserResponse;
import com.medibook.medibookservice.service.UserService;
import jakarta.validation.Valid;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
public class AuthController {

    private static final Logger log = LoggerFactory.getLogger(AuthController.class);

    private final UserService userService;

    public AuthController(UserService userService) {
        this.userService = userService;
    }

    /**
     * POST /api/v1/auth/register
     */
    @PostMapping("/register")
    public ResponseEntity<ApiResponse<UserResponse>> register(
            @Valid @RequestBody RegisterRequest request) {
        log.info("Registration request: {}", request.getEmail());
        UserResponse userResponse = userService.registerUser(request);
        ApiResponse<UserResponse> response = ApiResponse.success(
                "User registered successfully", userResponse);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    /**
     * POST /api/v1/auth/login
     */
    @PostMapping("/login")
    public ResponseEntity<ApiResponse<AuthResponse>> login(
            @Valid @RequestBody LoginRequest request) {
        log.info("Login request: {}", request.getEmail());
        AuthResponse authResponse = userService.login(request);
        ApiResponse<AuthResponse> response = ApiResponse.success(
                "Login successful", authResponse);
        return new ResponseEntity<>(response, HttpStatus.OK);
    }
}
```

22.3 URL Structure Cheat Sheet

URL anatomy

```
http://localhost:8080/api/v1/auth/register
\_________________/\____/\___/\________/
       Host         Context  Class   Method
       (port)       path     prefix  path
                    (yml)    @Req... @Post...
```

22.4 HTTP Status Code Standards

Code	Meaning	When to Use
200 OK	Success	GET, PUT, login
201 Created	Resource created	POST register
204 No Content	Success, no body	DELETE
400 Bad Request	Validation error	Bad input
401 Unauthorized	Not logged in	Missing/invalid token
403 Forbidden	No permission	Wrong role
404 Not Found	Resource missing	Wrong ID
409 Conflict	Duplicate	Email exists
500 Server Error	Code bug	Unhandled error

22.5 Initial SecurityConfig (testing only)

For initial testing, we use a permissive security config. We'll harden this in section 26 once JWT is in place.

config/SecurityConfig.java (initial)

```
package com.medibook.medibookservice.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
        return http.build();
    }
}
```

WARNING. Real bug we hit: I wrote **public HttpSecurity securityFilterChain(...)** by mistake. The return type must be **SecurityFilterChain**, and the final call must be **return http.build();** (Builder pattern). The build() method converts HttpSecurity into SecurityFilterChain.

PART VI — Authentication (JWT)

23. JWT Theory & Flow

23.1 What is JWT?

JSON Web Token (JWT) is an open standard (RFC 7519) for securely transmitting information between parties. It's:

- **Compact:** small enough to be sent as URL parameter or HTTP header
- **Self-contained:** contains all necessary user info
- **Signed:** can't be tampered without invalidating signature
- **Stateless:** server doesn't store sessions - JWT contains everything

23.2 JWT Structure

JWT anatomy

```
A JWT has three parts separated by dots:

    eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJqb2hu... . AbC123XyzSig
    \__________________/   \_________________/   \__________/
         HEADER                PAYLOAD            SIGNATURE
       (algorithm)          (user claims)       (verified by
                                                 secret key)

  Base64-encoded   Base64-encoded   HMAC-SHA256 of
  JSON object      JSON object      header+payload+secret
```

23.3 JWT Payload Example

Decoded JWT payload

```
{
  "sub": "john@example.com",    <- Subject (user identifier)
  "userId": 1,                  <- Custom claim
  "role": "PATIENT",            <- Custom claim
  "fullName": "John Doe",       <- Custom claim
  "iat": 1731782100,            <- Issued at (Unix timestamp)
  "exp": 1731868500             <- Expires at (24 hours later)
}
```

TIP. Visit *https://jwt.io* and paste a JWT — it shows the decoded header, payload, and verifies the signature. Great learning tool.

23.4 Complete JWT Authentication Flow

JWT request lifecycle

```
1. Client: POST /api/v1/auth/login { email, password }
            |
            v
2. Server: BCrypt.matches(password, hashedPassword)
            |
            v
3. Server: JwtService.generateToken(user)
            - Claims: { sub: email, userId, role }
            - Sign with secret key
            - Set expiry (24 hours)
            |
            v
4. Server -> Client: 200 OK { token: "eyJ...AbC123" }
            |
            v
5. Client: Stores token (localStorage / secure storage)
            |
            v
6. Client: GET /api/v1/users/me
   Headers: Authorization: Bearer eyJ...AbC123
            |
            v
7. Server: JwtAuthenticationFilter intercepts
   - Extract token from header
   - Verify signature with secret
   - Check expiration
   - Load user from DB
   - Set authentication in SecurityContext
            |
            v
8. Server: Controller method runs
   - @AuthenticationPrincipal provides logged-in user
   - @PreAuthorize checks role
            |
            v
9. Server -> Client: 200 OK { user data }
```

23.5 Why Stateless = Scalable

Stateful Sessions (Old)	Stateless JWT (Modern)
Server stores session in memory/Redis	Server stores nothing
Cannot easily scale horizontally	Add servers freely (no shared state)
Sticky sessions / session replication	Any server can serve any request
Logout = delete server session	Logout = client discards token (or use blocklist)
Doesn't work easily across services	Works seamlessly in microservices

24. JwtService (Token Generation & Validation)

24.1 Add JWT dependencies to pom.xml

pom.xml

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
```

After adding, right-click **pom.xml » Maven » Reload Project**.

24.2 JwtService Code

security/JwtService.java

```
package com.medibook.medibookservice.security;

import com.medibook.medibookservice.entity.User;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Service
public class JwtService {

    private static final Logger log = LoggerFactory.getLogger(JwtService.class);

    @Value("${medibook.jwt.secret}")
    private String jwtSecret;

    @Value("${medibook.jwt.expiration-ms}")
    private long jwtExpirationMs;

    public String generateToken(User user) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("userId", user.getId());
        claims.put("role", user.getRole().name());
        claims.put("fullName", user.getFullName());

        Date now = new Date();
        Date expiry = new Date(now.getTime() + jwtExpirationMs);

        return Jwts.builder()
                .claims(claims)
                .subject(user.getEmail())
                .issuedAt(now)
                .expiration(expiry)
                .signWith(getSigningKey())
                .compact();
    }

    public String extractEmail(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public Long extractUserId(String token) {
        return extractClaim(token, claims -> claims.get("userId", Long.class));
    }

    public String extractRole(String token) {
        return extractClaim(token, claims -> claims.get("role", String.class));
    }

    public boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }

    public boolean isTokenValid(String token, String email) {
```

```
        try {
            String tokenEmail = extractEmail(token);
            return tokenEmail.equals(email) && !isTokenExpired(token);
        } catch (Exception e) {
            log.warn("Invalid JWT: {}", e.getMessage());
            return false;
        }
    }

    private <T> T extractClaim(String token, Function<Claims, T> resolver) {
        Claims claims = Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token)
                .getPayload();
        return resolver.apply(claims);
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Base64.getEncoder().encode(jwtSecret.getBytes());
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

25. Login Implementation

Login is already implemented in **UserServiceImpl.login()** (see section 21.3). Key points to remember:

- Use **passwordEncoder.matches(raw, hashed)** - never compare strings directly
- Same error for wrong email and wrong password (anti-enumeration)
- Check **user.isEnabled()** to support account disabling
- Generate JWT only after all checks pass
- Return token + minimal user info (no password hash!)

26. Protecting Endpoints with JwtAuthenticationFilter

26.1 Filter Chain Concept

Spring Security filter chain

26.2 CustomUserDetailsService

Spring Security needs to know how to load a user. We tell it: 'find by email in our users table'.

security/CustomUserDetailsService.java

```
package com.medibook.medibookservice.security;

import com.medibook.medibookservice.entity.User;
import com.medibook.medibookservice.repository.UserRepository;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Collections;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        User user = userRepository.findByEmail(email)
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " + email));

        return new org.springframework.security.core.userdetails.User(
                user.getEmail(),
                user.getPassword(),
                user.isEnabled(),
                true,    // accountNonExpired
                true,    // credentialsNonExpired
                true,    // accountNonLocked
                Collections.singleton(new SimpleGrantedAuthority("ROLE_" + user.getRole().name()))
        );
    }
}
```

WARNING. Important: Spring Security expects roles prefixed with **ROLE_**. We convert Role.ADMIN to "ROLE_ADMIN". Then @PreAuthorize("hasRole('ADMIN')") works (it automatically prepends ROLE_).

26.3 JwtAuthenticationFilter

security/JwtAuthenticationFilter.java

```
package com.medibook.medibookservice.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private static final Logger log = LoggerFactory.getLogger(JwtAuthenticationFilter.class);

    private final JwtService jwtService;
    private final CustomUserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService,
                                   CustomUserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request,
                                    @NonNull HttpServletResponse response,
                                    @NonNull FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        try {
            String token = authHeader.substring(7);   // skip "Bearer "
            String email = jwtService.extractEmail(token);

            if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails userDetails = userDetailsService.loadUserByUsername(email);

                if (jwtService.isTokenValid(token, email)) {
                    UsernamePasswordAuthenticationToken authToken =
                            new UsernamePasswordAuthenticationToken(
                                    userDetails, null, userDetails.getAuthorities());
                    authToken.setDetails(
                            new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                    log.debug("Authenticated: {}", email);
                }
            }
        } catch (Exception e) {
            log.warn("JWT auth failed: {}", e.getMessage());
```

```
        }

        filterChain.doFilter(request, response);
    }
}
```

26.4 Final SecurityConfig (JWT-aware)

config/SecurityConfig.java (final)

```
package com.medibook.medibookservice.config;

import com.medibook.medibookservice.security.CustomUserDetailsService;
import com.medibook.medibookservice.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfigurat
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableMethodSecurity   // enables @PreAuthorize annotations
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final CustomUserDetailsService userDetailsService;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter,
                          CustomUserDetailsService userDetailsService) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/auth/**").permitAll()
                        .requestMatchers("/actuator/**").permitAll()
                        .anyRequest().authenticated())
                .sessionManagement(s ->
                        s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authenticationProvider(authenticationProvider())
                .addFilterBefore(jwtAuthenticationFilter,
                        UsernamePasswordAuthenticationFilter.class);

        return http.build();
```

```
    }
}
```

26.5 Config Settings Explained

Setting	What it Does
csrf().disable()	CSRF protection is needed for stateful sessions; we use JWT instead
/auth/** permitAll	Login and register stay public
/actuator/** permitAll	Health checks stay public
anyRequest().authenticated()	Everything else needs JWT
SessionCreationPolicy.STATELESS	No server-side sessions - pure JWT
addFilterBefore(jwtAuthFilter, ...)	Our filter runs first in the chain
@EnableMethodSecurity	Enables @PreAuthorize annotations

27. Role-Based Access Control (RBAC)

27.1 UserController with Protected Endpoints

controller/UserController.java

```
package com.medibook.medibookservice.controller;

import com.medibook.medibookservice.dto.response.ApiResponse;
import com.medibook.medibookservice.dto.response.UserResponse;
import com.medibook.medibookservice.entity.User;
import com.medibook.medibookservice.mapper.UserMapper;
import com.medibook.medibookservice.service.UserService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;
    private final UserMapper userMapper;

    public UserController(UserService userService, UserMapper userMapper) {
        this.userService = userService;
        this.userMapper = userMapper;
    }

    /**
     * GET /api/v1/users/me
     * Returns the currently logged-in user's info.
     * Any authenticated user can access.
     */
    @GetMapping("/me")
    public ResponseEntity<ApiResponse<UserResponse>> getCurrentUser(
            @AuthenticationPrincipal UserDetails userDetails) {
        User user = userService.findByEmail(userDetails.getUsername());
        UserResponse response = userMapper.toResponse(user);
        return ResponseEntity.ok(ApiResponse.success("User details retrieved", response));
    }

    /**
     * GET /api/v1/users/admin-only
     * Only ADMIN users can access.
     */
    @GetMapping("/admin-only")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ApiResponse<String>> adminOnly() {
        return ResponseEntity.ok(ApiResponse.success(
                "Access granted", "Welcome, Admin!"));
    }
}
```

27.2 @PreAuthorize Expressions

Expression	Meaning
hasRole('ADMIN')	User must have ROLE_ADMIN
hasAnyRole('ADMIN', 'DOCTOR')	User has one of these roles
hasAuthority('READ_USER')	Permission-based (more fine-grained)

Expression	Meaning
isAuthenticated()	Just logged in
#userId == authentication.principal.userId	User can only access own data
@securityService.isOwner(#id, authentication)	Custom check

27.3 New Concepts in UserController

Annotation/Class	Purpose
@AuthenticationPrincipal	Auto-injects the currently logged-in user
UserDetails	Spring Security's user representation
userDetails.getUsername()	Returns the email (in our case)
@PreAuthorize("hasRole('ADMIN')")	Method-level RBAC; 403 if check fails

PART VII — Testing & Errors

28. Testing with Postman — All Scenarios

28.1 Test 1 — Register a Patient (Happy Path)

POST http://localhost:8080/api/v1/auth/register

Request

```
Headers:
  Content-Type: application/json

Body (raw / JSON):
{
    "email": "john@example.com",
    "password": "Password123!",
    "fullName": "John Doe",
    "phoneNumber": "9876543210",
    "role": "PATIENT"
}
```

Expected response

```
HTTP 201 Created
{
    "success": true,
    "message": "User registered successfully",
    "data": {
        "id": 1,
        "email": "john@example.com",
        "fullName": "John Doe",
        "phoneNumber": "9876543210",
        "role": "PATIENT",
        "enabled": true,
        "createdAt": "2026-05-18T22:30:00"
    },
    "timestamp": "2026-05-18T22:30:00"
}
```

28.2 Test 2 — Login

POST http://localhost:8080/api/v1/auth/login

Request

```
Body:
{
    "email": "john@example.com",
    "password": "Password123!"
}
```

Expected response

```
HTTP 200 OK
{
    "success": true,
    "message": "Login successful",
    "data": {
        "token": "eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOjEsInJv...",
        "tokenType": "Bearer",
        "userId": 1,
        "email": "john@example.com",
        "fullName": "John Doe",
        "role": "PATIENT"
    },
    "timestamp": "..."
}
```

28.3 Test 3 — Access /users/me with Token

GET http://localhost:8080/api/v1/users/me

Request

```
Headers:
  Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySW...
```

TIP. In Postman: Authorization tab » Type: Bearer Token » paste token from login response (no quotes, no "Bearer " prefix - just the token).

28.4 Error Tests — Verify Exception Handler Works

#	Test Case	Expected
1	Register same email twice	409 Conflict — "User already exists with email..."
2	Invalid email format	400 Bad Request — validation error on email
3	Password too short / weak	400 Bad Request — validation error on password
4	Missing required field (role)	400 Bad Request — validation error
5	Multiple invalid fields	400 Bad Request — all errors in validationErrors map
6	Wrong password on login	401 Unauthorized — "Invalid email or password"
7	Non-existent email on login	401 Unauthorized — same as wrong password (security!)
8	Access /users/me without token	401/403
9	Access /users/me with expired token	401/403
10	PATIENT accessing /admin-only	403 Forbidden
11	ADMIN accessing /admin-only	200 OK

29. Every Error We Hit (and How We Fixed It)

Real errors faced during development of this project. Save this section - you will probably hit these too.

Error 1 — Public Key Retrieval is not allowed

Error

```
java.sql.SQLNonTransientConnectionException:
    Public Key Retrieval is not allowed
```

Cause: MySQL 8 uses caching_sha2_password authentication that requires fetching the public key from the server.

Fix: Add **allowPublicKeyRetrieval=true** to the JDBC URL in application-dev.yml:

Fixed URL

```
url: jdbc:mysql://localhost:3306/medibook_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetri
```

Error 2 — Access denied for user 'root'@'localhost'

Error

```
java.sql.SQLException:
    Access denied for user 'root'@'localhost' (using password: YES)
```

Cause: DB_PASSWORD environment variable not set or wrong.

Fix:

- Verify password works: **mysql -u root -p** in CMD
- In IntelliJ: Run Configurations » Modify options » Environment variables
- Add: **DB_PASSWORD=your_actual_password** (no quotes!)

Error 3 — YAML Parser Error

Cause: Wrong indentation in YAML. Most commonly: **logging:** indented under **spring:** by accident.

Fix rules:

- Use 2 spaces, NEVER tabs
- Root keys (spring:, server:, logging:) at column 1
- Same indent level = same hierarchy level
- Space after colon: "key: value"

Error 4 — Lombok Plugin Won't Install

Error

```
Plugin "Lombok" was not installed:
Request failed with status code 40x
```

Cause: Corporate firewall blocking IntelliJ marketplace.

Fix:

- Option A: Download plugin .zip from *plugins.jetbrains.com* on another network, install via Settings » Plugins » gear icon » Install from disk
- Option B (we chose this): Drop Lombok, use plain Java with manual getters/setters

Error 5 — Port 8080 Already in Use

Cause: Previous Spring Boot process didn't fully shut down.

Symptom: App starts on port 8081 instead of 8080 (DevTools picks next free).

Fix:

Kill old process

```
# In CMD as administrator:
netstat -ano | findstr :8080
# Note the PID (last number)
taskkill /PID 12345 /F
```

Error 6 — NoResourceFoundException

Error

```
NoResourceFoundException: No static resource auth/register
```

Cause: Forgot the context-path prefix. The URL was http://localhost:8081/auth/register but the actual endpoint is http://localhost:8081**/api/v1**/auth/register.

Fix: Always include the full context path. Remember the URL structure: host + context-path (from yml) + class prefix (@RequestMapping) + method path.

Error 7 — HttpRequestMethodNotSupportedException: GET not supported

Cause: Tried to access POST endpoint via browser URL bar. Browsers send GET.

Fix: Use Postman / IntelliJ HTTP Client / curl for POST/PUT/DELETE. Browser URL bar only does GET.

Error 8 — IntelliJ Returns Wrong Class Type

Bug

```
// BAD (wrong return type)
@Bean
public HttpSecurity securityFilterChain(HttpSecurity http) {
    http.csrf(...).authorizeHttpRequests(...);
    return http;
}
```

Fix

```
// GOOD (Builder pattern)
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) {
    http.csrf(...).authorizeHttpRequests(...);
    return http.build();   // <-- .build() converts HttpSecurity to SecurityFilterChain
}
```

Error 9 — 500 Internal Server Error on Register

Cause: Multiple possible. Check the IntelliJ console for the real exception.

Common causes:

- Role sent as lowercase "patient" instead of "PATIENT"
- @EnableJpaAuditing missing on main class
- users table missing a column

Error 10 — Test Class Runs Instead of Main App

Symptom: Console shows **Started MedibookServiceApplicationTests** with test class, not the main app.

Cause: Clicked the green play next to **MedibookServiceApplicationTests** (in src/test/...) instead of **MedibookServiceApplication** (in src/main/...).

Fix: Always open **src/main/java/.../MedibookServiceApplication.java** and click the green play there. Top-right dropdown should say **MedibookServiceApplication** (no "Tests" suffix).

30. Common Mistakes Cheat Sheet

Category	Common Mistake	Prevention
Java	Using == to compare strings	Use .equals() or Objects.equals()
Java	NullPointerException from method chain	Use Optional or null-check
Spring	Field injection with @Autowired	Use constructor injection (final fields)
Spring	Forgetting @Transactional on writes	Add @Transactional to all writing service methods
Spring	Returning entity directly	Always map to DTO first
JPA	Forgetting no-arg constructor	JPA requires public/protected no-arg constructor
JPA	ddl-auto: create-drop in prod	Use validate in prod; update only in dev
JPA	Not closing transactions	@Transactional handles this; use it!
Security	Storing plain text passwords	Always BCrypt with strength 10+
Security	Hardcoding secrets in code	Use environment variables
Security	Same error reveals if email exists	Use same message for both wrong-email and wrong-passwo
Security	JWT secret too short	Use at least 256 bits (32+ chars)
REST	Returning 200 for created resources	Use 201 Created
REST	Returning HTML error pages	Always JSON; configure global exception handler
REST	Verbs in URL: /getUsers	Use nouns + HTTP methods: GET /users
YAML	Using tabs	Always 2 spaces; never tabs
YAML	Wrong indentation	Verify hierarchy visually
MySQL	Default ddl-auto creates tables	Set explicit ddl-auto in yml
MySQL	Forgetting to create database	CREATE DATABASE medibook_db; first
Git	Committing application.yml with passwords	Use .gitignore + env vars
Git	Committing target/ folder	.gitignore should include target/

PART VIII — Reference

31. Production Best Practices Checklist

[OK] Password Hashing BCrypt with strength 10+. Never store plain text passwords.

[OK] DTO Pattern Never expose entities directly. Separate request/response DTOs.

[OK] Stateless Authentication JWT tokens instead of sessions - scales horizontally.

[OK] Environment Variables Secrets (DB password, JWT secret) never in code.

[OK] Profile-Based Config Different settings for dev/staging/prod with same code.

[OK] Layered Architecture Controller -> Service -> Repository. Each layer one job.

[OK] Constructor Injection Final fields, no @Autowired. Immutable, testable, fail-fast.

[OK] Bean Validation @Valid + @NotBlank etc. - automatic input validation.

[OK] Global Exception Handler All errors caught centrally, no stack-trace leaks.

[OK] No Email Enumeration Same error for wrong-email vs wrong-password (OWASP).

[OK] Connection Pooling HikariCP with proper pool sizes (min/max idle).

[OK] Audit Fields @CreatedDate, @LastModifiedDate auto-fill timestamps.

[OK] Database Indexing @Index on email column - fast lookups.

[OK] Health Endpoint Actuator /health for load balancer probes.

[OK] SOLID Principles Interface segregation, dependency inversion, etc.

[OK] Transactions @Transactional for writes; readOnly=true for reads.

[OK] ddl-auto: validate (prod) Never auto-modify production schema.

[OK] CORS Configuration Configure allowed origins (will do in frontend phase).

[OK] HTTPS Only in Production AWS ALB terminates SSL; backend gets HTTPS.

[OK] Logging Strategy INFO for events, WARN for unusual, ERROR for failures.

[OK] JSR-310 (java.time) Use LocalDateTime, never Date or Calendar.

[OK] @JsonInclude(NON_NULL) Don't return null fields in JSON.

[OK] Pagination from Day 1 findAll(Pageable) - prevent loading 1M rows.

32. Interview Talking Points

Use this section when explaining the project in interviews. Each topic is phrased as you might say it in conversation.

32.1 "Tell me about a project"

I built MediBook - a doctor appointment booking system. Phase 1 is a Spring Boot monolith with JWT authentication and role-based access for three user types: patient, doctor, admin. I followed a monolith-first approach because the domain wasn't fully understood yet - the plan is to split into microservices in later phases once the boundaries become clear.

32.2 "How do you handle authentication?"

I use JWT tokens for stateless authentication. On login, I verify the BCrypt hashed password and generate a signed JWT containing user id, email, and role as claims. Subsequent requests carry the token in the Authorization Bearer header. A custom OncePerRequestFilter intercepts every request, validates the token signature and expiry, and sets the SecurityContext. This is stateless - any instance can serve any request, which scales horizontally.

32.3 "How do you handle errors?"

I use a global exception handler annotated with @RestControllerAdvice. Custom exceptions like ResourceNotFoundException, ResourceAlreadyExistsException, and BadRequestException each map to specific HTTP status codes (404, 409, 400). MethodArgumentNotValidException is caught to return field-level validation errors in a consistent envelope. A catch-all handler logs full stack traces internally but returns a generic message to clients - this prevents internal details leaking to attackers.

32.4 "How do you ensure security?"

Several layers. Passwords are BCrypt hashed - never stored in plain text. Database credentials and JWT secret come from environment variables, never in code. CSRF is disabled because we use JWT (not session cookies). For login, I return the same error message for wrong email and wrong password - this prevents email enumeration attacks per OWASP guidelines. Role-based access uses @PreAuthorize for method-level security.

32.5 "How do you decide on architecture?"

I followed Martin Fowler's monolith-first principle. The package structure (controller, service, repository, entity, dto) is microservices-ready - when we split later, each domain (user, doctor, slot, booking, notification) becomes a separate service with the same internal structure. I applied SOLID principles: interface segregation by separating UserService interface from impl, dependency inversion by injecting the interface, single responsibility per layer.

32.6 "How would you optimize for performance?"

I added a unique index on the email column - login lookups become O(log n) instead of O(n). For 1 million users that's the difference between 5 seconds and 2 milliseconds. I use HikariCP connection pooling with tuned pool sizes. I use @Transactional(readOnly=true) for read operations - Hibernate skips dirty checking. For pagination I use Spring Data's Pageable interface which generates LIMIT/OFFSET SQL automatically.

32.7 "Why JWT over sessions?"

JWT is stateless - servers don't store session data. This means: any server instance can serve any request (horizontal scaling), no shared session storage needed (Redis cluster cost), works seamlessly across

microservices, mobile-friendly (no cookies), and faster (no DB lookup per request). Trade-off: tokens can't be invalidated easily - I'd add a blocklist or refresh token pattern in production.

33. Glossary of Terms

Term	Definition
JPA	Java Persistence API - standard for ORM in Java
ORM	Object-Relational Mapping - maps Java objects to DB rows
Hibernate	Most popular JPA implementation
JDBC	Java Database Connectivity - low-level DB API
DTO	Data Transfer Object - object used to transfer data between layers
POJO	Plain Old Java Object - simple class with fields/getters/setters
Bean	An object managed by Spring's IoC container
IoC	Inversion of Control - container creates objects, not you
DI	Dependency Injection - container provides dependencies
JWT	JSON Web Token - signed token for stateless auth (RFC 7519)
BCrypt	One-way password hashing algorithm; slow by design
RBAC	Role-Based Access Control - permissions based on roles
REST	Representational State Transfer - architectural style for APIs
CRUD	Create, Read, Update, Delete - basic data operations
HTTP	HyperText Transfer Protocol
HTTPS	HTTP over TLS/SSL (encrypted)
CORS	Cross-Origin Resource Sharing - browser security mechanism
CSRF	Cross-Site Request Forgery - attack on stateful sessions
XSS	Cross-Site Scripting - injecting JS into web pages
OWASP	Open Web Application Security Project - security standards body
SOLID	Five OO design principles (S,O,L,I,D)
DRY	Don't Repeat Yourself - code reuse principle
KISS	Keep It Simple, Stupid - simplicity principle
YAGNI	You Aren't Gonna Need It - avoid premature features
TDD	Test-Driven Development - write tests first
CI/CD	Continuous Integration / Continuous Deployment
LTS	Long-Term Support - stable, multi-year support version
IDE	Integrated Development Environment

Term	Definition
API	Application Programming Interface
JSON	JavaScript Object Notation - data format
YAML	YAML Ain't Markup Language - config format
XML	eXtensible Markup Language - older config format
LLD	Low-Level Design - detailed class/component design
HLD	High-Level Design - architectural overview
Maven	Build tool and dependency manager for Java
pom.xml	Maven Project Object Model - dependencies and config
Hibernate Auditing	Auto-populate @CreatedDate/@LastModifiedDate
HikariCP	Default Spring Boot connection pool (fastest)
Actuator	Spring Boot module providing /health, /metrics, etc.

34. What's Next: Phase 2 Onwards

Phase 2 — Doctor Module

Doctor entity, specializations, admin CRUD, public browsing, pagination/sorting/filtering. Add Swagger/OpenAPI for API documentation.

Phase 3 — Slot Management

Slot generation based on work hours, admin configuration of slot rules. Implement slot conflict detection.

Phase 4 — Booking System

Patient bookings with concurrency handling (pessimistic locking). Prevent double-booking when 2 patients click 'book' simultaneously.

Phase 5 — Async Notifications

Email confirmations via Kafka (local) / AWS SQS+SNS (cloud). Implement CQRS pattern for command (book) and query (view bookings).

Phase 6 — Microservices Split

Break monolith into: auth-service, doctor-service, slot-service, booking-service, notification-service. Add Eureka (service discovery), Spring Cloud Gateway (API gateway), Feign Client (inter-service calls), Resilience4j (circuit breaker).

Phase 7 — Frontend (React)

Three React portals with TypeScript: Patient (browse + book), Doctor (manage appointments), Admin (manage everything). Bootstrap UI.

Phase 8 — Containerization

Dockerize each service, multi-container setup with Docker Compose. Multi-stage Docker builds for smaller images.

Phase 9 — CI/CD

GitHub Actions pipeline: test, build, push image, deploy. Bonus: Jenkins as alternative CI/CD.

Phase 10 — AWS Deployment (Free Tier)

S3 + CloudFront for frontend, Elastic Beanstalk + RDS for backend, Lambda + API Gateway for one service (serverless demo), SQS + SNS for messaging, ALB + Auto Scaling Group for high availability.

Phase 11 — Observability

Logs (ELK stack), metrics (Prometheus + Grafana), distributed tracing (Sleuth + Zipkin or OpenTelemetry).

Phase 1 Complete!

You've built a production-grade foundation.
Same architecture used by Netflix, Spotify, Uber, and major fintech apps.

Keep building. The journey continues.